

Multi-Threaded Video Analysis and Performance Monitoring Software Using C++ and OpenCV

Submitted by: Krishna (24BCE0592), Anushka Sarkar (24BCI0049)

Submitted to: Dr. Arthi M Ma'am

GITHUB REPOSITORY - https://github.com/Krishna-24-24/Spectra

INDEX

1. Introduction 3
2. Abstract 3
3. Project Objectives 4
4. Literature Review 4
5. Methodology 6
5.1 System Architecture and Processing Pipeline 6
5.2 Producer-Consumer Model 8
5.3 Thread Pool Design 9
5.4 Synchronization and Frame Ordering 9
5.5 Frame Analysis and Event Detection 10
5.6 Computational Flow 11
5.7 OS Concepts Applied 12
6. Sample Implementation 12
6.1 Development Environment 12
6.2 GUI Design and Application Architecture 13
6.3 Optimization Techniques 13
6.4 Test Dataset 14
6.5 GUI Walkthrough Example 14
7. Experimental Methodology 15
8. Results and Performance Evaluation 16
9. Testing and Validation 23
10. Conclusion and Future Scope 25
11. References 27

Multi-Threaded Video Analysis and Performance Monitoring Software Using C++ and OpenCV

1. Introduction

Manually reviewing long recorded video footage to locate moments of interest is slow, tedious, and error-prone. This challenge is common across everyday scenarios, including CCTV and security footage, campus surveillance recordings, traffic monitoring feeds, industrial process monitoring, and general long-duration recordings. A reviewer who has to watch hours of largely static footage to find the few seconds that actually matter is spending time inefficiently, and is also more likely to miss short but significant events due to fatigue. A single-threaded review or processing tool also uses only one CPU core at a time, leaving the remaining cores on a modern multi-core machine idle.

This project implements a complete desktop application, developed in C++17 with a Qt 6 graphical interface, that automates the first pass of this review process. The user opens a video file, and the software analyzes it frame by frame using OpenCV [1] to identify timestamps containing significant visual activity: motion, high activity, scene changes, and sudden visual changes. Because frame-level analysis is an independent, repeatable operation across thousands of frames, the workload is well suited to parallel execution. The application therefore implements a multi-threaded processing engine built around a **producer-consumer architecture** and a **fixed-size thread pool**, and it includes a built-in benchmark that measures how much faster the parallel version really is.

The system deliberately avoids machine learning and deep learning. It does not claim to identify people, weapons, or threats. Instead it reports observable, explainable visual signals so that a human reviewer can quickly jump to the relevant portion of a long video and make the final judgment. This keeps the system lightweight, hardware-cheap, and explainable, without requiring GPUs or training data.

2. Abstract

This project presents a C++17 desktop application for automated, multi-threaded analysis of video footage. A single producer thread decodes the video and pushes per-frame tasks into a bounded, thread-safe task queue (std::mutex and std::condition_variable); a fixed pool of worker threads takes tasks from the queue and, using OpenCV, converts each frame to grayscale, applies a Gaussian blur, differences it against the previous frame, thresholds the result, and measures the changed area and brightness histogram. Every frame carries a unique, increasing frame ID, and a result manager re-orders the results so that the detected events are identical regardless of the number of threads. An event detector turns the ordered per-frame measurements into four kinds of events: Significant Motion, High Activity, Scene Change, and Sudden Visual Change.

A Qt 6 graphical interface lets the user browse and play a video, choose single-threaded or multi-threaded analysis with 1, 2, 4 or 8 workers, watch live progress, click an event to seek the video to that moment, stop the analysis safely at any time, and export an events CSV, a benchmark CSV and an annotated video. A built-in benchmark repeats the same analysis for different thread counts and reports execution time, throughput (FPS), speedup, parallel efficiency, CPU use and memory, together with four graphs and an automatically generated written interpretation.

The complete system was built, tested and measured. On a Windows 11 laptop with 12 logical processors, analyzing a 900-frame 1280x720 test video, the benchmark (run before the final corrections described in Section 8.5) measured **9.42 s with 1 thread (95.6 FPS)** and **6.88 s with 4 threads (130.8 FPS)**, a speedup of **1.37x** with 34.2% parallel efficiency; 8 threads were slower than 1 thread (0.70x). The report explains why: video decoding is not parallel (Amdahl's Law), and OpenCV's own internal threads compete with the application's thread pool. On a second, 2-processor test machine the same program reached **1.86x** with 2 threads. Detection was checked against a test video with known events: four of the five events were found on the Windows laptop with the original scene-change rule; after the rule was corrected, all five were found on the Linux test machine, with identical results for 1, 2, 4 and 8 threads. The corrected build was not benchmarked again on the Windows laptop. Seven automated test programs cover the queue, pool, analyzer, detector, ordering, whole engine and the GUI; all seven pass on the Linux test machine.

3. Project Objectives

1. Automate detection of significant visual events (motion, scene changes) — Verified against a test video whose events are known in advance, to ensure accuracy.

2. Accelerate video processing using a multi-threaded C++ engine — Measured via execution time and FPS across thread counts (1, 2, 4, 8 threads).

3. Provide a responsive GUI for video navigation and live status — The GUI must never freeze during processing; progress, FPS, workers and events update in real time, and Stop works safely.

4. Benchmark and evaluate parallel efficiency across different thread counts — Using the standard formulas: Speedup = T(1) / T(n), Efficiency = Speedup / n x 100%.

5. Make the system reliable and verifiable — Friendly handling of bad input, safe cancellation, no fabricated results, and automated tests that can be re-run.

4. Literature Review

We reviewed existing tools, textbooks, and studies on multithreaded video and frame processing to inform the design of the producer-consumer pipeline, the thread pool, and the synchronization strategy used in this project.

4.1 Manual Review (Baseline)

Manual review, a human watching footage in real time or faster to spot events of interest, is the standard baseline practice against which this project is compared. It is time-consuming and prone to observer fatigue, and it serves as the main benchmark for measuring the gains delivered by automation.

4.2 OS Concurrency Foundations

Silberschatz, Galvin & Gagne (2018) — Operating System Concepts, 10th ed., Wiley — [2] provides the foundational thread synchronization theory used in this project's thread pool: process/thread models, mutual exclusion, condition variables, and the (bounded-buffer) producer-consumer problem. This textbook underpins the correctness arguments made for the task queue design in Section 5.

4.3 C++ Threading Practice

Anthony Williams (2019) — C++ Concurrency in Action, 2nd ed., Manning Publications — [3] informs the concrete C++ implementation of the thread pool and task queue, including guidance on std::thread lifecycle management, std::mutex and std::condition_variable usage. This source guided the RAII-based join-on-destruction pattern used in the ThreadPool class.

4.4 Prior Work on Multithreaded Video and Media Processing

OpenCV Multithreaded Video Processing Sample (video_threaded.py, OpenCV project) — [4] The official OpenCV samples repository includes a reference implementation that organizes frame capture and processing into a parallel pipeline using a thread pool and a pending-task queue, explicitly to keep video playback smooth by overlapping I/O with computation. This directly shaped the frame-task abstraction used in this project.

Development of Multi-process Video Stitching in AVM Applications Based on OpenCV (ICMIR 2018) — [5] This work partitions a video-surveillance pipeline into distinct stages assigned to separate threads, reporting a throughput improvement of roughly 2.12 times for a four-camera setup. It validated keeping frame decoding (the producer) architecturally separate from frame analysis (the consumers).

Multithreading with OpenCV-Python to Improve Video Processing Performance (Syed, 2018) — [6] This article shows that dedicating separate threads to frame capture and to frame display, in addition to processing, substantially increases achievable throughput, since I/O-bound work can proceed independently of CPU-bound image processing.

Parallel Detection for Efficient Video Analytics at the Edge (Wu, Liu & Kompella, 2021) — [7] This paper distributes detection work across parallel execution paths with a scheduler to reduce disparity in processing rates, analogous to the queue-based task distribution used by the thread pool in this project.

Real-Time Multiobject Tracking Based on Multiway Concurrency (Gong et al., 2021) — [8] This study reinforces the importance of imposing a fixed limit on the number of worker threads, rather than allowing unbounded concurrency, to keep resource usage predictable under concurrent multi-stream video processing.

Multithread Processing of Video Frames (frame/slice-level parallel decoding literature) — [9] This work identifies a specific inefficiency: a worker thread that finishes its assigned units early sits idle while others are still working. This directly informed the decision to use a shared, dynamically-drained task queue rather than statically dividing frames among workers in advance.

Amdahl (1967) — Validity of the single processor approach to achieving large scale computing capabilities — [10] states that the speedup of a program is limited by its sequential part. In this project the sequential part is video decoding by the single producer thread, and this law is used in Section 8 to explain why speedup flattens.

Across these sources, three recurring themes stand out: (1) separating I/O-bound decoding from CPU-bound analysis consistently improves throughput; (2) a bounded, reusable pool of worker threads avoids both the overhead of unbounded thread creation and the idling caused by static work division; and (3) explicit synchronization is required wherever a shared queue or shared result buffer is accessed by more than one thread.

4.5 Current Gap

Existing tools are either purely manual and slow, or rely on machine-learning models that are accurate but GPU-dependent and harder to explain or justify. This project fills the gap between the two: a lightweight, CPU-only, explainable approach that uses classical frame differencing rather than a trained model, while using multithreading to make use of the available CPU cores, and that measures its own parallel performance honestly.

5. Methodology

This section explains how the video analysis pipeline is structured and parallelized.

5.1 System Architecture and Processing Pipeline

The end-to-end pipeline follows a layered flow:

- Input video → Video decoder (producer) → Task queue → Thread pool of workers (frame analysis) → Result manager (re-ordering by frame ID) → Event detector → Reports and GUI output → Performance report.

A video such as the 900-frame test video used in this project is decoded frame by frame, and each frame is converted into an independent processing task that can be handed to any available worker thread. The program is divided into two parts that never mix: the **processing engine** (library spectra_core, plain C++17 and OpenCV, no Qt) and the **Qt user interface**. Because the engine has no GUI code, it can be unit-tested on its own and also driven from a command-line

tool (spectra_cli). Figure 1 shows the architecture, and Table 1 lists the components and their responsibilities.

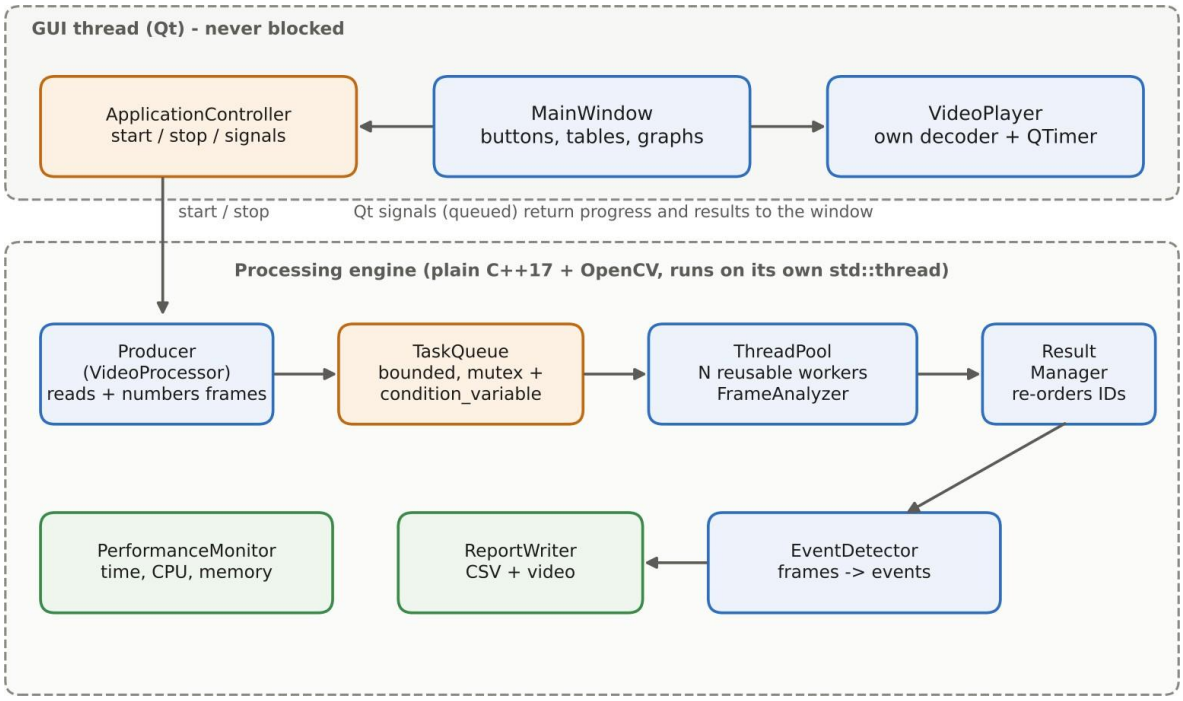


Figure 1: System architecture. The window and the video player live in the GUI thread; the engine runs on its own thread and reports back through Qt signals.

Table 1: Components and their responsibilities

Component	Class	Responsibility
GUI layer	MainWindow	Browse, settings, live status, events table, performance tab, benchmark, graphs
Application controller	ApplicationController	Starts the engine on a background thread, owns the Stop flag, converts engine results into Qt signals
Video player	VideoPlayer, VideoView	Play / pause / stop / seek with its own decoder, independent of the analysis
Producer	VideoProcessor	Reads frames, numbers them, attaches the previous frame, pushes tasks
Task queue	TaskQueue	Bounded producer-consumer buffer (mutex + condition variables)
Thread pool	ThreadPool	N reusable worker threads that run the frame analysis

Component	Class	Responsibility
Frame analyzer	FrameAnalyzer	Grayscale, blur, difference, threshold, contours, histogram
Result manager	ResultManager	Stores results by frame ID and releases them in order
Event detector	EventDetector	Groups per-frame measurements into events
Performance monitor	PerformanceMonitor	Wall time, CPU usage, memory
Benchmark manager	BenchmarkManager	Runs 1/2/4/8 threads, computes speedup and efficiency, writes the interpretation
Report writer	ReportWriter	Events CSV, benchmark CSV, annotated video

5.2 Producer-Consumer Model

The video reader acts as the **producer**: it reads frames sequentially from the file and pushes them, wrapped as tasks, onto a thread-safe task queue. A configurable number of worker threads act as **consumers**: each worker repeatedly takes the next available task from the queue, analyzes the frame, and stores the result. The queue has a fixed capacity (the larger of 6 and 3 x the number of workers), so if the workers are slower than the reader the reader waits instead of filling the memory with decoded frames. This is the classic bounded-buffer problem from operating systems.

The code excerpts in this report are abridged for readability (accessors, helper members and some bookkeeping are omitted); they follow the structure of the project source files.

```
struct FrameTask {
    std::uint64_t frameId;      // unique, increasing number
    double        timestampSec;
    cv::Mat       frame;        // current frame
    cv::Mat       previous;     // previous frame (not copied)
};

class TaskQueue {
public:
    explicit TaskQueue(std::size_t capacity);
    bool push(FrameTask&& task);   // blocks while full
    bool pop(FrameTask& out);      // blocks while empty
    void close();                  // no more frames; drain queue
    void cancel();                 // stop now; wake everyone
private:
    std::mutex mutex_;
    std::condition_variable notEmpty_, notFull_;
    std::queue<FrameTask> queue_;
    std::size_t capacity_;
    bool closed_ = false, canceled_ = false;
};
```

Two different shutdown calls exist on purpose. close() is used when the video simply ends: workers finish all frames that are already queued. cancel() is used when the user presses Stop: queued frames are thrown away and every sleeping thread is woken immediately.

5.3 Thread Pool Design

A new thread is not created for every frame, because thread creation and destruction carry non-trivial overhead when repeated thousands of times. Instead, a fixed pool of worker threads (1, 2, 4 or 8, selectable by the user) is created once at the start of the analysis. Each worker runs a loop that takes tasks from the shared queue until the queue is closed and empty. The destructor cancels the queue and joins every thread (the RAII pattern), so no thread can be left running.

```
void ThreadPool::workerLoop()
{
    FrameTask task;
    while (queue_.pop(task)) {          // sleeps while there is no work
        try {
            handler_(task);             // analyze frame + store result
        } catch (...) {                 // a bad frame must never kill a worker
        }
        task = FrameTask();             // release frame memory
    }
}

ThreadPool::~ThreadPool() { queue_.cancel(); join(); }   // never blocks forever
```

5.4 Synchronization and Frame Ordering

The task queue and the result store are both shared across threads and must be protected against concurrent modification. Access to the queue is guarded by a std::mutex, and two std::condition_variable objects make idle threads **sleep instead of busy-waiting**: notEmpty_ wakes a worker when a frame arrives, and notFull_ wakes the producer when a slot becomes free. This prevents race conditions, lost or duplicated tasks, and deadlocks.

Because workers may finish frames in a different order than they were dispatched (a visually complex frame can take longer than a simple one), every frame is given a unique, increasing frame ID at extraction time. The ResultManager stores results in a map keyed by this ID and only releases the **next expected ID** to the event detector, so events are always computed in the original video order.

```
void ResultManager::store(const FrameResult& result)
{
    std::lock_guard<std::mutex> lock(mutex_);
    pending_[result.frameId] = result;       // may arrive out of order
    drainLocked();
}

void ResultManager::drainLocked()            // release results in order
{
    for (;;) {
        auto it = pending_.find(nextId_);
        if (it == pending_.end()) break;
        detector_.onFrame(it->second);       // always in frame order
        pending_.erase(it);
        ++nextId_;
    }
}
```

Why each task carries the previous frame. Frame differencing needs frame n and frame n-1. If a worker had to wait for another worker to provide the previous frame, the workers could not run in parallel. The producer therefore attaches the previous frame to every task. cv::Mat is reference-counted, so this copies a pointer, not the pixels. Every task is fully independent, and the answer does not depend on how many threads are used. This is verified by an automatic test (Section 9).

5.5 Frame Analysis and Event Detection

Each frame-processing task performs the following steps using OpenCV:

- Shrink the frame if it is wider than 1280 pixels (HD video is analyzed at its own size).
- Convert the frame to grayscale.
- Apply a 9x9 Gaussian blur to reduce sensor and compression noise.
- Compute the absolute difference between the current and the previous frame.
- Apply a threshold of 25 (a pixel counts as "changed" if it differs by more than 25 of 255).
- Measure the **changed ratio** (fraction of changed pixels) and the largest connected changed region using contours.
- Compute a 64-bin brightness histogram and the Bhattacharyya distance [11] to the previous frame's histogram, and the change in average brightness.

The EventDetector then labels each frame (at most one label per frame, checked in this order) and groups neighboring labeled frames into events:

Table 2: Event rules in the final (corrected) build

Order	Label	Rule (all values are configurable)
1	Sudden Visual Change	average brightness jumps by 40 or more levels (a flash)
2	Scene Change	10% or more of the pixels changed AND histogram distance of 0.40 or more (a hard cut)
3	High Activity	10% or more of the pixels changed
4	Significant Motion	0.5% or more of the pixels changed

Chronology of the scene-change rule. Table 2 shows the rule used in the final build. The first Windows build, which produced Table 5 and Table 6, used a stricter scene-change rule: at least **25%** of the pixels changed and a histogram distance of at least **0.50**. After the Windows run missed the scene change at 11 s, the rule was corrected to **10%** and **0.40** (Sections 8.2 and 8.5). The corrected rule was tested on the Linux machine and in the automatic tests; it has not been run again on the Windows laptop.

- Labelled frames closer than **0.75 s** are merged into **one** event (otherwise 90 frames of one movement would give 90 events).
- Motion bursts shorter than **3 frames** are ignored as noise.
- A motion event becomes High Activity if at least half of its frames were intense.
- **Score (0 to 1):** motion = peak changed ratio / 0.25 (capped at 1); scene change = changed ratio; sudden change = brightness jump / 100.

For example, an entry produced by the program on the test video is: Timestamp 00:00:16.000, Type High Activity, Score 0.54, Duration 3.03 s. No machine learning is used; the detection logic is a small, explainable set of image-difference computations, which makes the algorithm easy to justify and defend.

5.6 Computational Flow

1. Load the video, read its metadata (resolution, FPS, frame count, duration) and check that the first frame can be decoded.
2. Create the task queue and, in multi-threaded mode, the thread pool; start the performance monitor.
3. The producer decodes frames sequentially and enqueues tasks with unique frame IDs.
4. Worker threads take tasks, analyze the frames and store the results.
5. The result manager re-orders results by frame ID and the event detector builds the list of events.
6. Measure execution time, FPS, CPU usage and memory; compute speedup and efficiency (benchmark).

7. Show the results, events and performance in the GUI and export the CSV reports and annotated video.

5.7 OS Concepts Applied

Concurrency and Synchronization — Mutexes and condition variables protect the shared task queue and the result map from race conditions. Without them, two threads reading or writing at the same moment could corrupt the data or lose tasks, causing frames to be skipped or processed twice. Atomic variables are used for the Stop flag and for the progress counters.

Bounded-Buffer (Producer-Consumer) Problem — The queue has a fixed capacity, the producer sleeps when it is full and consumers sleep when it is empty. This decouples the speed of reading from the speed of processing without letting memory grow without limit.

CPU Scheduling and Resource Reuse — A fixed-size thread pool reuses worker threads instead of creating an OS thread per frame. Thread creation and destruction are expensive at the operating-system level, so reuse keeps the threads alive in a wait state until work is available. Using more worker threads than the machine can run at the same time causes extra context switches and competition for the same cores; this effect is observed in Section 8.

Thread Lifecycle and Safe Termination — Every thread is joined before the program continues. Stop sets an atomic flag, the queue wakes all sleeping threads, workers finish the frame they hold, and the pool joins. Nothing is killed forcefully.

System Monitoring — Execution time is measured with std::chrono::steady_clock. CPU usage is computed as process CPU time divided by (elapsed time x number of logical processors), using the Windows GetProcessTimes call (and getrusage on Linux). Memory is the process working set sampled during the run (GetProcessMemoryInfo on Windows), and the highest value is reported.

Amdahl's Law — Only the frame analysis is parallel; video decoding by the single producer is sequential. The maximum possible speedup is therefore limited, which is observed in the results.

6. Sample Implementation

This section describes how the design in Section 5 is realized as working software. The application was built incrementally: a minimal GUI first, then single-threaded analysis, the thread-safe queue, the thread pool, event detection, the results dashboard, and finally benchmarking, error handling, tests and documentation.

6.1 Development Environment

- Programming language: C++17
- GUI framework: Qt 6 Widgets [12] (installed with vcpkg [13])
- Image processing and video decoding: OpenCV 4 (core, imgproc, videoio; installed with vcpkg)

- Concurrency: std::thread, std::mutex, std::condition_variable, std::atomic
- Build system: CMake; compiler: MSVC (Visual Studio 2022); target platform: Windows 10/11
- Tests: seven plain C++ test programs registered with CTest; the engine and tests were also built with GCC on Linux, which is where the automated tests were run
- Version control: Git / GitHub [14] (https://github.com/Krishna-24-24/Spectra)
- Graphs: drawn directly with QPainter (no Qt Charts library is needed)

6.2 GUI Design and Application Architecture

The GUI is kept strictly separate from the processing engine. All video analysis runs on background threads; the Qt GUI thread is never blocked. Progress, live FPS, active worker count and events are sent to the window using Qt signals and slots, which are safe for cross-thread communication (custom data types are registered with qRegisterMetaType). The window is organized in two columns:

- **1. Video Input** - Browse Video button and the file name, resolution, frame rate, frame count, duration and size.
- **2. Video Player** - picture, seek bar, Play/Pause, Stop and current/total time.
- **3. Analysis Settings** - single-threaded or multi-threaded mode, 1/2/4/8 workers, motion / scene change / activity checkboxes, option to save the processed video, START ANALYSIS and STOP.
- **4. Live Status** - progress bar, frames processed, processing speed, active workers, events found and a status line.
- **Detected Events tab** - table of Timestamp, Event Type, Score and Duration; clicking a row seeks the video to that moment.
- **Performance & Benchmark tab** - time, FPS, CPU, memory, speedup and efficiency of the last analysis; benchmark controls (1/2/4/8 threads, repeats); the benchmark table; and a written summary generated from the measured numbers.
- **Graphs tab** - four graphs: processing time, FPS, speedup (with the ideal line) and efficiency (with the 100% line).

While an analysis or benchmark runs, the settings are disabled and only STOP is active. The video player keeps working because it uses its own decoder.

6.3 Optimization Techniques

- Bounded, reusable thread pool instead of per-frame thread creation, to minimize thread-management overhead.
- Condition-variable blocking on the queue, so idle workers sleep instead of busy-waiting.
- Frame IDs used purely for result ordering, which decouples completion order from output order and avoids barriers between workers.
- The previous frame is attached to every task by reference, so tasks are independent and no pixels are copied.

- Grayscale conversion and blur before differencing reduce noise and the cost of the comparison.
- Progress updates are throttled (about every 100 ms) so the Qt event loop is not flooded by the workers.
- The analysis size was increased from 640 to 1280 pixels width after measurements showed that the per-frame work at 640 pixels was too small compared with decoding (Section 8.4).
- OpenCV's own internal threading is limited to one thread during an analysis, so the only parallelism measured is the application's own thread pool (Section 8.5).

6.4 Test Dataset

A 30-second test video, sample_events.mp4 (1280x720, 30 fps, 900 frames, H.264, 0.47 MB), was generated by a script included in the project (tools/make_sample_video.py). Because the video is generated, the time of every event is known exactly, so the detector can be checked against a ground truth instead of being trusted blindly:

Table 3: Ground truth of the test video

Time (s)	What happens in the video	Expected event
5 - 8	one white box slides across the picture	Significant Motion
11	hard cut to a different (checkerboard) scene	Scene Change
16 - 19	22 small boxes jump around the whole picture	High Activity
22	one single white flash frame	Sudden Visual Change
25 - 28	a yellow box slides back across the picture	Significant Motion

6.5 GUI Walkthrough Example

The user clicks **Browse Video**, selects sample_events.mp4, chooses Multi-threaded mode with 4 workers and presses **START ANALYSIS**. The progress bar and the frame counter advance while the analysis runs, and the events appear in the Detected Events table. Clicking a row, for example High Activity at 00:00:16.000, moves the video to that moment. Figure 2 shows the application on the Windows laptop after the analysis and the benchmark had finished.

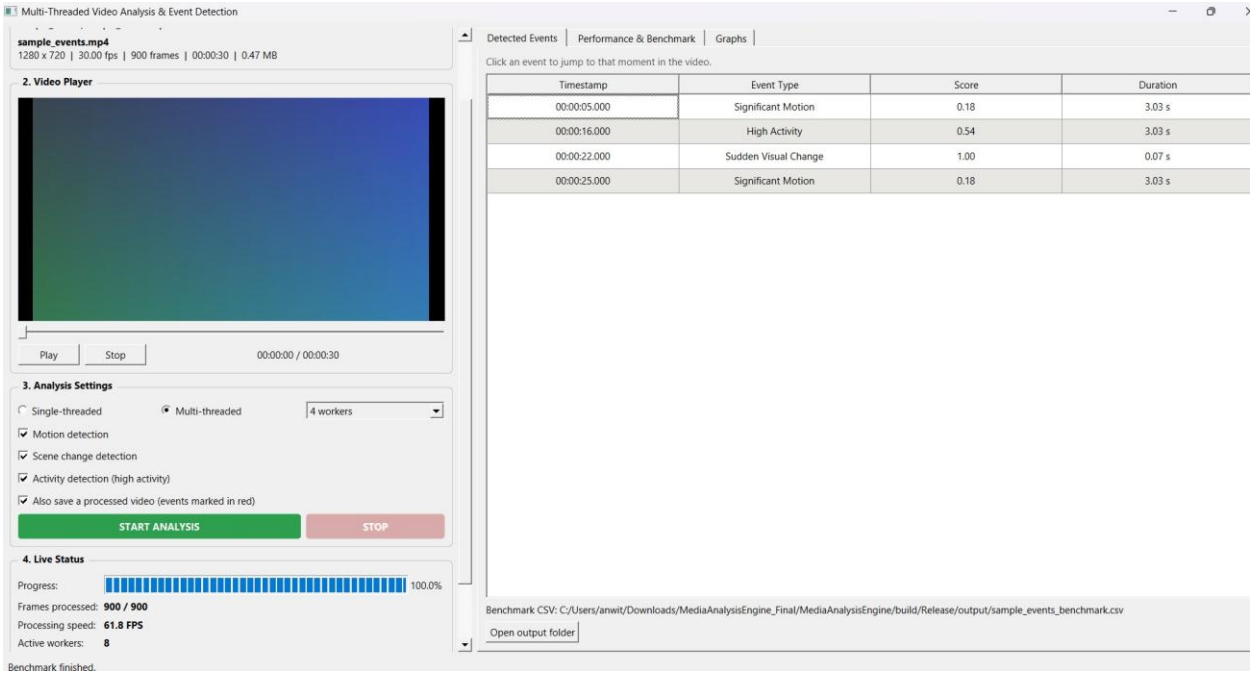


Figure 2: The main window on the Windows laptop: video information, player, settings and live status on the left; the Detected Events table and the saved report paths on the right.

7. Experimental Methodology

7.1 Timing Procedure

Execution time is measured with std::chrono::steady_clock. The timer starts when the analysis begins (after the video has been opened and checked) and stops when the result manager has processed the last frame, so file selection and GUI start-up are not counted. The single-thread baseline is a true sequential loop (read, analyze, store) with **no queue and no pool**, so the speedup shows what the producer-consumer design and the thread pool really gained.

7.2 Warm-up and Repeated Trials

Before the measured runs the benchmark performs one warm-up run on the first 300 frames, which is not recorded; it lets the operating system cache the video file so that the first measured run is not unfairly slow. Each thread count (1, 2, 4, 8) is then measured **twice** (the number of repeats can be changed in the GUI) and the mean is reported, which smooths out operating-system scheduling noise.

7.3 Derived Metrics

- **FPS** = frames processed / execution time.
- **Speedup** = T(1) / T(n), comparing each configuration with the single-threaded baseline.
- **Parallel efficiency** = Speedup / n x 100%, showing how close the system gets to ideal linear scaling.
- **CPU usage** = process CPU time / (elapsed time x logical processors) x 100%.

- **Memory** = highest working-set size sampled during the run.

7.4 Ground-Truth Validation

The detected events are compared with the known events of the test video (Table 3). A detection counts as correct if its type matches and its start time is within a small tolerance (0.5 s) of the ground truth. This check measures detection accuracy independently of the performance measurements, and the same comparison is run automatically as part of the test suite.

7.5 Test Machines

Table 4: Machines used for the measurements

	Windows laptop (user machine)	Linux test machine
Operating system	Windows 11	Linux
Logical processors	12	2
Build	MSVC, Visual Studio 2022, Release; GUI application	GCC, Release; command-line benchmark and tests
Libraries	Qt 6 and OpenCV 4 from vcpkg	Qt 6 and OpenCV 4.6 from the distribution
Video	sample_events.mp4 (900 frames, 1280x720)	same file

8. Results and Performance Evaluation

All numbers in this section were measured by the application itself; none are estimated or projected. Sections 8.1 and 8.2 report the run on the Windows laptop (a build made before the final corrections) exactly as it came out, including its weaknesses; Section 8.3 reports the second machine, which ran the final corrected code; Sections 8.4 and 8.5 explain what was learned.

8.1 Benchmark on the Windows Laptop

Table 5 lists the benchmark measured on the Windows laptop with the build that existed before the corrections of Section 8.5. Speedup, efficiency and FPS follow the formulas of Section 7.3 and were recalculated from the times for this report; they agree with the values printed by the program.

Table 5: Benchmark results on the Windows laptop (12 logical processors, 900 frames, mean of 2 runs per row)

Threads	Time (s)	FPS	Speedup (x)	Efficiency (%)	CPU (%)	Memory (MB)
1	9.417	95.6	1.00	100.0	33.5	390.0
2	6.975	129.0	1.35	67.5	70.4	414.3
4	6.882	130.8	1.37	34.2	71.2	401.0
8	13.457	66.9	0.70	8.7	40.5	402.8

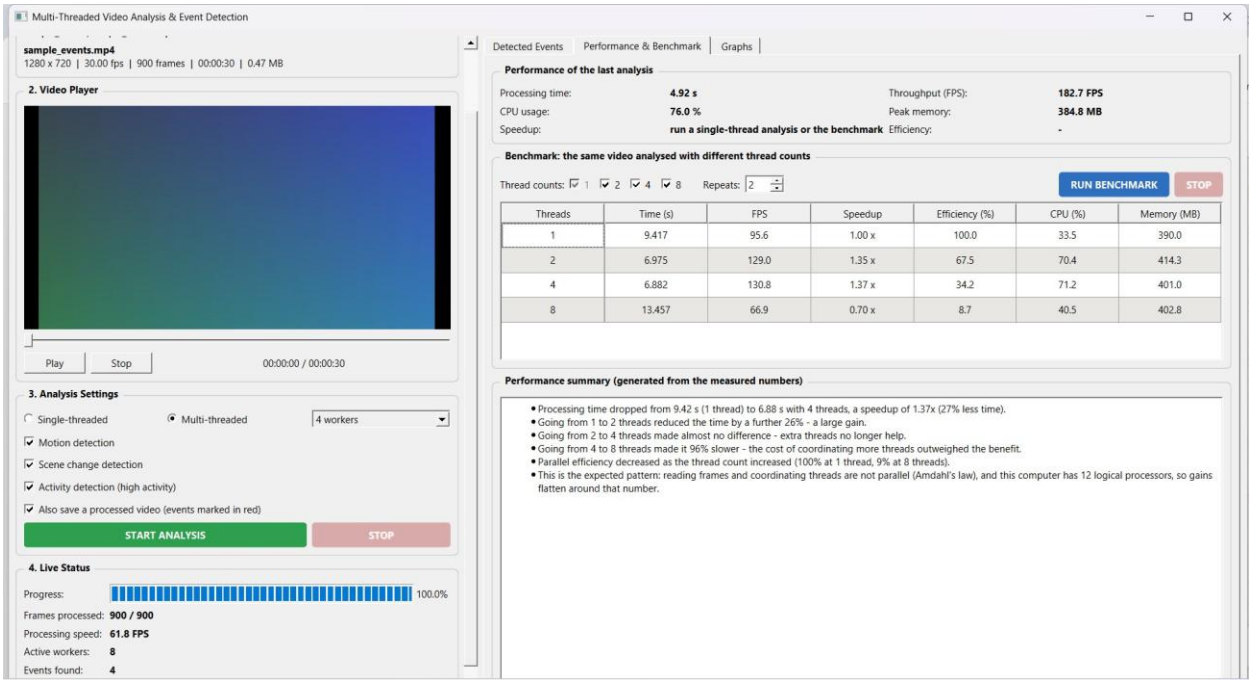


Figure 3: Performance & Benchmark tab on the Windows laptop: measured table and the automatically generated summary.

The application generated the following summary from these numbers (copied from the screen, Figure 3):

- Processing time dropped from 9.42 s (1 thread) to 6.88 s with 4 threads, a speedup of 1.37x (27% less time).
- Going from 1 to 2 threads reduced the time by a further 26% - a large gain.
- Going from 2 to 4 threads made almost no difference - extra threads no longer help.
- Going from 4 to 8 threads made it 96% slower - the cost of coordinating more threads outweighed the benefit.
- Parallel efficiency decreased as the thread count increased (100% at 1 thread, 9% at 8 threads).

Note on the wording of the summary. The on-screen sentence "96% slower" refers to execution time: 13.457 s / 6.882 s = 1.955, so the 8-thread run took about 95.5% longer than the 4-thread run (the program rounds this to 96%). Throughput changed by a different amount: 66.9 FPS / 130.8 FPS = 0.51, so about 49% fewer frames were processed per second. Both percentages describe the same measurement and must not be confused. Figure 4 shows the same results graphically.

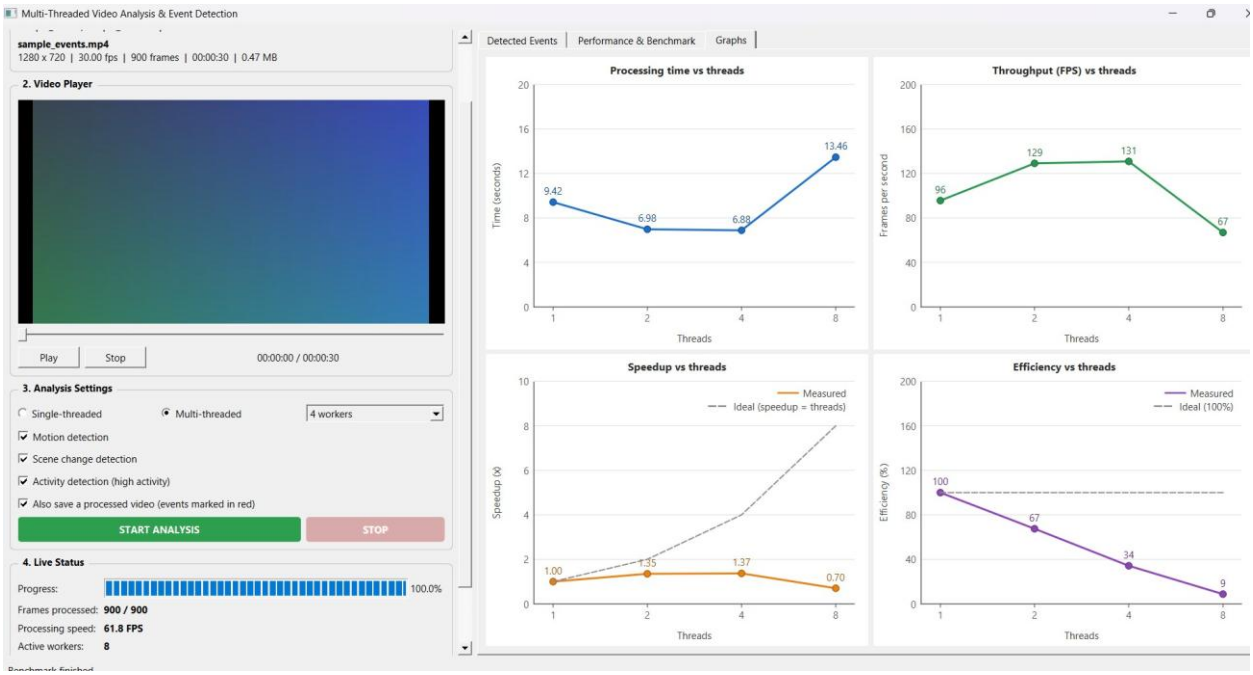


Figure 4: Graphs tab on the Windows laptop: processing time, throughput, speedup (with the ideal line) and efficiency (with the 100% line).

8.1.1 Discussion of the Windows Results

- **Speedup is real but small.** Two threads gave 1.35x and four threads 1.37x. Beyond 2 threads the throughput stays at about 130 FPS. This is consistent with Amdahl's Law: the single producer thread must decode every frame, and once it cannot deliver frames faster than a certain rate, adding workers cannot help. The plateau at about 130 FPS suggests that this rate was reached.
- **8 threads were slower than 1 thread (0.70x).** The laptop has 12 logical processors, so 8 workers should fit, yet the run took about 96% longer than with 4 threads (13.457 s versus 6.882 s). We believe this is oversubscription: OpenCV has its own internal thread pool, and each of the 8 workers calls OpenCV functions, so many more software threads than processors compete for the CPU and the cost of switching between them exceeds the useful work. The measurements are consistent with this explanation: the CPU usage was 70-71% at 2 and 4 threads (about 8-9 of the 12 processors busy) while the speedup was only 1.35-1.37x, meaning that much of the processor time did not produce extra frames. However, oversubscription alone would normally keep the processors busy, whereas the CPU usage at 8 threads fell to 40.5%. This means that threads spent much of the run waiting, or that the processor was slowed (for example by power management or heat). No separate experiment isolated the cause, so the oversubscription explanation remains a hypothesis.
- **The 1-thread baseline was not strictly one core.** The 1-thread run used 33.5% of the CPU. With 12 logical processors, 33.5% corresponds to 0.335 x 12 = about 4.0 processors' worth of CPU time, whereas one fully occupied logical processor would show 1/12 = about 8.3%. The baseline was a sequential loop in the application's own code, so the extra CPU time must

have come from other threads of the same process: most likely OpenCV's internal worker threads (which parallelize some image operations on their own) and the threads of the video decoder, and possibly the user-interface thread. The measurement does not separate these contributions, so their individual shares were not determined. The effect is that the baseline ran faster than a true single core would, which makes the measured speedup look smaller (Section 8.5).

- **Run-to-run variation is large on a laptop.** A separate single analysis with 4 workers, run before the benchmark, took 4.92 s (182.7 FPS, CPU 76%, peak memory 384.8 MB), while the benchmark's 4-thread mean was 6.88 s, which is about 40% longer (6.88 / 4.92 = 1.40). A difference of this size between runs of the same configuration (possible causes: laptop power management, heat, background activity; not isolated) is much larger than the 5-10% we assumed, and it is a reason why the benchmark repeats and averages each configuration. The single timings in Table 5 should therefore be read as indicative rather than exact.
- **Memory** stayed between about 390 and 414 MB for all thread counts. The bounded queue keeps the amount of buffered frames constant, so memory does not grow with the number of threads. The value includes the Qt window and the video player.

8.2 Event Detection on the Windows Laptop

Table 6 and Figure 5 compare the events found on the Windows laptop with the ground truth of the test video (Table 3). The timestamps of all detected events are exact to the frame.

Table 6: Detected events versus ground truth on the Windows laptop (4 of 5 found)

Ground truth	Detected on Windows	Score	Duration	Result
5 - 8 s Significant Motion	00:00:05.000 Significant Motion	0.18	3.03 s	Correct
11 s Scene Change	-	-	-	**Missed**
16 - 19 s High Activity	00:00:16.000 High Activity	0.54	3.03 s	Correct
22 s Sudden Visual Change	00:00:22.000 Sudden Visual Change	1.00	0.07 s	Correct
25 - 28 s Significant Motion	00:00:25.000 Significant Motion	0.18	3.03 s	Correct

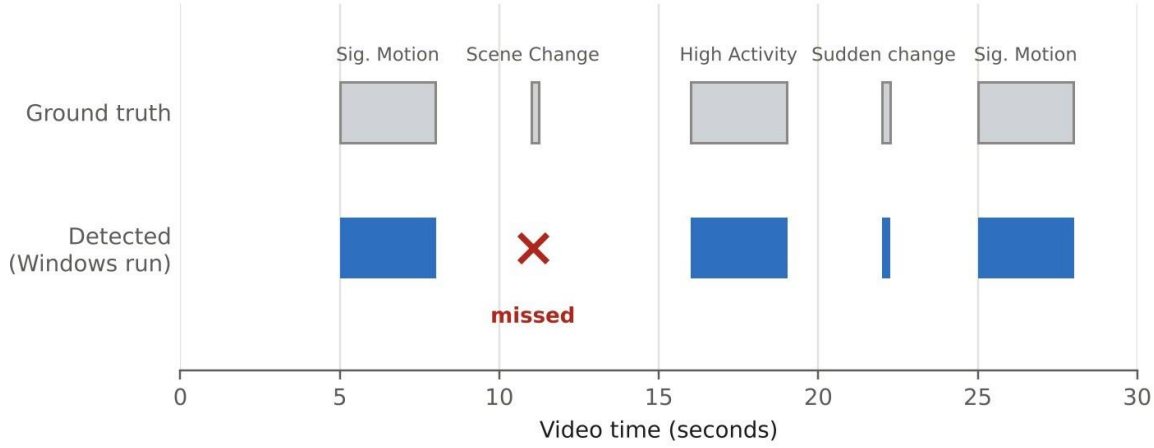


Figure 5: Ground truth (gray) versus the events detected on the Windows laptop (blue). The scene change at 11 s was missed.

Why was the scene change missed? The original scene-change rule (used in the Windows build that produced Tables 5 and 6) required at least 25% of the pixels to change **and** a brightness-histogram distance of at least 0.50. On the Linux test machine the cut at 11 s measured 27.96% changed pixels and a distance of 0.71, so the first condition passed with only a small margin. The Windows video decoder produces slightly different pixel values for the same file, so on the laptop at least one of the two conditions was not met. This is a useful lesson: a rule that works with a thin margin on one machine can fail on another. The other four events were detected correctly on the laptop.

Correction. Examination of all frames of the test video showed that only the real cut and the flash ever have a histogram distance above 0.15, so the scene-change rule was corrected (Table 2) to **10% changed pixels and histogram distance 0.40**, which leaves wide margins on both conditions. After the change all five events are found (Section 8.3) and the automatic test passes. This corrected rule has not yet been re-measured on the Windows laptop.

8.3 Validation on a Second Machine

The command-line version of the same engine was run on a Linux machine with 2 logical processors, using the final code, on the same video file. All five events were found at the expected times (Table 8), and the event list was identical for 1, 2, 4 and 8 threads (checked automatically). Table 7 lists the benchmark of this machine.

Table 7: Benchmark on the 2-processor Linux test machine (mean of 2 runs per row; no GUI, so memory is lower)

Threads	Time (s)	FPS	Speedup (x)	Efficiency (%)	CPU (%)	Memory (MB)
1	8.198	109.8	1.00	100.0	57.5	96.4
2	4.396	204.7	1.86	93.2	98.1	114.4
4	4.439	202.7	1.85	46.2	98.3	139.5
8	4.584	196.3	1.79	22.4	95.3	131.7

Table 8: Events found with the corrected rules (5 of 5)

Detected on Linux (any thread count)	Score	Duration
00:00:05.000 Significant Motion	0.18	3.03 s
00:00:11.000 Scene Change	0.28	0.03 s
00:00:16.000 High Activity	0.51	3.03 s
00:00:22.000 Sudden Visual Change	1.00	0.07 s
00:00:25.000 Significant Motion	0.18	3.03 s

Figure 6 compares the speedup of the two machines. With only 2 processors, the best possible speedup is 2x, and the measured 1.86x with 2 threads (93% efficiency) shows that the thread pool and queue add very little overhead. More threads than processors give no further gain, and the small decrease at 8 threads (1.79x) is the cost of the extra context switching. The two machines together show the same law from two sides: speedup is limited by the number of processors and by the sequential decoding, whichever comes first.

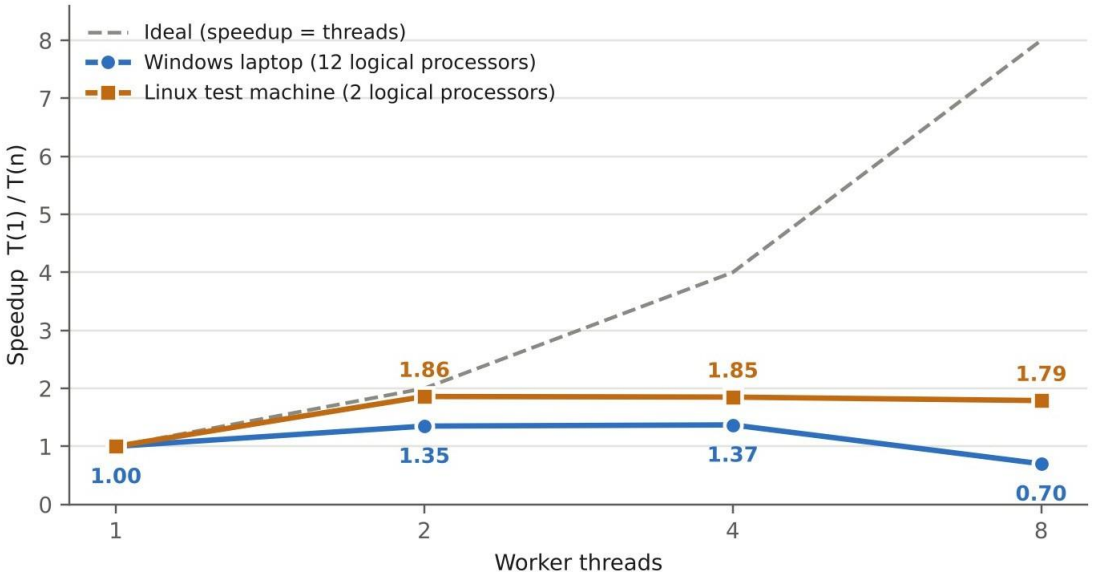


Figure 6: Measured speedup on both machines compared with the ideal. The Windows laptop has far more processors but a smaller speedup (see Sections 8.1 and 8.5).

8.4 Design Experiment: Amount of Work per Frame

The first prototype in this project (a console program with simulated frames, reported in the earlier stage of this work) reached only 1.23x because each task was tiny and synchronization cost dominated. The same effect appeared again with real video. When frames were first shrunk to 640 pixels before analysis, the work per frame was so small compared with decoding that two threads gave almost no gain; analyzing the frames at their native size (1280 pixels) multiplied the useful work per task. The measurement on the Linux test machine is given in Table 9:

Table 9: Effect of the work per frame on the speedup (Linux test machine)

Analysis width	1 thread (s)	2 threads (s)	Speedup with 2 threads	Events found
640 px (shrunk)	2.739	2.569	1.07x	5
1280 px (native HD)	7.692	4.180	1.84x	5

The detected events were the same in both cases, but only the heavier workload made parallelism worthwhile. This is why the application analyzes HD frames at their native size. It confirms the principle that the amount of work per task must be large compared with the cost of coordinating threads.

8.5 Limitations and Changes Made After the Windows Measurements

Table 10: Problems found by the Windows measurements and the corrections

Finding on the Windows laptop	Change made to the program	Verified where
Scene change at 11 s missed (rule had a thin margin)	Rule relaxed to 10% changed pixels and histogram distance 0.40	5 of 5 events on the Linux machine and in the automatic test; not yet re-measured on the laptop
1-thread baseline used about 4 cores (33.5% CPU) and 8 threads were slower than 1: OpenCV's own threads compete with the thread pool	OpenCV is limited to one internal thread during an analysis (restored afterwards), so only the application's pool provides parallelism	No change in results and all tests pass on the Linux machine (2 processors); the benchmark has not been re-run on the 12-processor laptop

These corrections (Table 10) follow from the measurements, but the Windows benchmark in Table 5 was taken **before** them. Table 5 therefore shows the program as it behaved at that time, and re-running the benchmark with the final build on the laptop is expected to show a smaller 1-thread CPU usage and a different speedup. This is stated openly so that the reported numbers are not mistaken for the results of the final build.

8.6 Output Files

After an analysis the program writes three files into the output folder next to the executable (the paths are shown in the window, Figure 2):

```
sample_events_events.csv
  timestamp,event_type,score,duration
  00:00:05.000,Significant Motion,0.18,3.03
  00:00:11.000,Scene Change,0.28,0.03
  00:00:16.000,High Activity,0.51,3.03
  ...
sample_events_benchmark.csv
  threads,time,fps,speedup,efficiency,cpu_percent,memory_mb
  1,8.157,110.3,1.00,100.0,57.1,98.2
  2,4.336,207.6,1.88,94.1,98.8,124.8
sample_events_processed.avi
  the original video with a red frame and the event name drawn
  while an event is active (MJPG, needs no codec pack)
```

The CSV excerpts above were produced by the program (final code, Linux test machine) and show the format (they come from a separate run, so their timings differ slightly from Table 7); on the Windows laptop the same files were saved under build/Release/output/ as shown in Figure 2.

9. Testing and Validation

Unit and integration tests — Seven test programs are built with the project and run with CTest (ctest --output-on-failure). All seven pass in the Linux development environment; the first six also build without the GUI libraries.

Table 11: Automated tests

Test	What it checks
taskqueue	First-in-first-out order; one producer and four consumers deliver every task exactly once; the queue is bounded and the producer sleeps when it is full; close() delivers the remaining items; cancel() wakes sleeping consumers and blocked producers
threadpool	1000 tasks on 4 workers use at most 4 threads (threads are reused) and really run at the same time; an exception inside a task does not stop the worker; destroying an idle pool never hangs
frameanalyzer	Identical frames give zero change; a moving box gives the expected changed area; a brightness jump is measured; HD frames are shrunk correctly; 8 threads analyzing the same input give identical answers
eventdetector	Continuous motion becomes one event, not one per frame; nearby bursts merge and distant ones stay separate; short bursts are ignored as noise; High Activity, Scene Change and Sudden Visual Change are classified correctly; switching a detector off works
resultmanager	Results delivered in a random order by 4 threads are processed in frame order and give the same events as ordered input; gaps after a cancel do not lose results
engine	Real video files: identical events for 1, 2, 4 and 8 threads; maxFrames; cancel returns promptly; missing file, non-video file and invalid thread counts give friendly errors; the real sample video matches its ground truth (5 of 5); both CSV formats; speedup/efficiency maths and the generated interpretation; a short real benchmark
gui_smoke	The real main window run off-screen: error on a missing file and on starting without a video; load the video; seek; run an analysis (5 events); click an event (the player jumps to 16 s); Stop during an analysis; run a benchmark (4 rows, 4 graphs filled)

Manual test on Windows — The application was built with MSVC and run on the Windows laptop. Loading a video, playing it, running a multi-threaded analysis, the live progress display, the events table, the performance panel, the benchmark with table, written summary and four graphs, and the saving of the CSV files and the processed video were all observed working (Figures 2 to 4).

Error handling — Each of the following produces a clear message instead of a crash:

Table 12: Error handling

Situation	Behavior
File does not exist, or is not a video	Message that the file could not be opened as a video; nothing starts
Video whose first frame cannot be decoded	Message that the video cannot be decoded
Thread count outside 1-64	Message asking for a valid thread count
No detector selected / no video chosen	Message asking the user to choose one
Fewer than two thread counts in the benchmark	Message asking for at least one count besides 1 (1 is always measured)
Output folder or file cannot be written	Message with the reason; the analysis results stay on screen
Video stops decoding early	Warning shown with the result
Exception inside a worker thread	Caught; the worker keeps running

Stop / Cancel — Stop sets an atomic flag; the producer stops reading, the queue drops pending frames and wakes every thread, workers finish the frame they hold and the pool is joined. The automatic tests verify that a canceled run returns promptly with a partial result, and that the window is usable again afterwards. After a stop, nothing is saved and the window says so.

Memory — The memory use measured on the Windows laptop was about 385-414 MB for all thread counts (Table 5), so it does not grow with the number of threads. Resources are managed with RAII (threads are joined in destructors, locks are scope-bound, frames are reference-counted), which prevents leaks by construction. No dedicated leak-detection tool was run on Windows for this report.

10. Conclusion and Future Scope

10.1 Conclusion

This project delivers a complete C++17, Qt 6 and OpenCV desktop application that automatically finds segments of significant activity in a video, using a producer-consumer pipeline and a thread pool. The system keeps frame order correct despite out-of-order parallel completion (the events are identical for 1, 2, 4 and 8 threads), keeps the GUI responsive through signal and slot communication, stops safely on request, and contains a benchmark that measures and visualizes execution time, throughput, speedup, efficiency, CPU use and memory. It applies the operating-system concepts of mutual exclusion, condition synchronization, the bounded-buffer problem, thread reuse and safe termination, and it is verified by seven automated tests.

The measurements gave honest and instructive results. On a 12-processor Windows laptop the best speedup was 1.37x with 4 threads, and 8 threads were slower than one. On a 2-processor machine the speedup was 1.86x with 2 threads. The measurements showed that the speedup is limited by sequential video decoding (Amdahl's Law), by the amount of work per frame, and by

oversubscription caused by OpenCV's own threads. The Windows run also exposed a detection rule with a thin margin, which was corrected. The final build has not yet been re-benchmarked on the laptop, and this is stated in Section 8.5.

Table 13: Objectives and outcomes

Objective	Outcome
1. Automatic detection of events	Achieved: 4 of 5 known events on the Windows laptop; 5 of 5 after the corrected rule (Linux machine and automatic test)
2. Faster processing with multiple threads	Achieved with limits: 1.37x on the 12-processor laptop, 1.86x on the 2-processor machine
3. Responsive GUI with live status and safe Stop	Achieved: verified manually on Windows and by the GUI test
4. Benchmark of parallel efficiency	Achieved: table, four graphs and generated explanation, saved to CSV
5. Reliable and verifiable	Achieved: friendly errors, safe cancel, seven automated tests, no fabricated results

10.2 Future Scope

- Re-run the benchmark on the final build on several machines and longer, higher-resolution videos to see how the speedup grows when the work per frame is larger.
- Decode several segments of the video in parallel (separate decoders per time range), which removes the sequential decoding limit of the single producer.
- Extend the thread pool with lock-free or wait-free queue variants to reduce synchronization overhead at high thread counts.
- Choose the number of workers automatically from the number of processors and a short calibration run.
- Use adaptive thresholds learned from the first seconds of a video instead of fixed values, so that rules do not depend on a thin margin.
- Port the frame-analysis pipeline to GPU acceleration (CUDA/OpenCL) for very high-resolution footage.
- Optionally add lightweight object classification (for example YOLO) after motion detection to identify what moved; this is explicitly outside the current scope and would be an opt-in layer on top of the explainable detectors.
- Support multi-camera and multi-stream ingestion with the same producer-consumer architecture. The engine is already separated into a reusable library with a command-line front end, which makes this and batch processing straightforward.

11. References

[1] Itseez. (2020). Open Source Computer Vision Library. https://opencv.org

[2] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.

[3] Williams, A. (2019). C++ Concurrency in Action (2nd ed.). Manning Publications.

[4] OpenCV Project. video_threaded.py - Multithreaded Video Processing Sample. https://github.com/opencv/opencv

[5] Development of Multi-process Video Stitching in AVM Applications Based on OpenCV. Proceedings of the International Conference on Mechatronics and Intelligent Robotics (ICMIR 2018).

[6] Syed, N. R. (2018). Multithreading with OpenCV-Python to Improve Video Processing Performance.

[7] Wu, Y., Liu, L., & Kompella, R. (2021). Parallel Detection for Efficient Video Analytics at the Edge. arXiv:2107.12563. https://arxiv.org/abs/2107.12563

[8] Gong, X., Le, Z., Wu, Y., & Wang, H. (2021). Real-Time Multiobject Tracking Based on Multiway Concurrency. Sensors, 21(3), 685. https://doi.org/10.3390/s21030685 (PMC7864016).

[9] Multithread Processing of Video Frames (frame/slice-level parallel decoding literature).

[10] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. AFIPS Spring Joint Computer Conference, 483-485.

[11] Bhattacharyya, A. (1943). On a measure of divergence between two statistical populations defined by their probability distributions. Bulletin of the Calcutta Mathematical Society, 35, 99-109.

[12] The Qt Company. Qt 6 Documentation (Signals and Slots; Threads and QObjects). https://doc.qt.io

[13] Microsoft. vcpkg - C++ package manager. https://github.com/microsoft/vcpkg

[14] Project repository: https://github.com/Krishna-24-24/Spectra